1. List All Docker Images

To list all the images on your system:

```
docker images
```

- This shows the **repository**, **tag**, **image ID**, **creation date**, and **size** of all images.

If you want a more detailed list (in JSON format):

```
docker images --format "{{json .}}"
```

2. List All Docker Containers

To list all **containers** (running, stopped, or exited):

```
docker ps -a
```

- This lists all containers, including their status (`Exited`, `Running`, etc.).

3. List Running Containers

To list only the **currently running containers**:

```
docker ps
```

- This command shows details like the container ID, image used, uptime, and ports.

4. List Running Images

Docker does not have a direct command to list images used by running containers, but you can use the following command to achieve this:

```
docker ps --format '{{.Image}}'
```

This lists the image names used by all running containers.

If you need more detailed information about running images, combine `docker ps` and `docker inspect`:

```
docker ps --format '{{.ID}} {{.Image}}'
```

Summary of Commands

Command	Purpose
`docker images`	List all images stored locally.
`docker ps`	List running containers.
`docker ps -a`	List all containers (running and stopped).
`docker ps --format '{{.Image}}'`	List images used by currently running containers.

1. Stop Running Containers

Command to Stop a Specific Container

```
docker stop <container_id_or_name>
```

- Replace `<container_id_or_name>` with the container's ID or name, which you can find using:`bash`

```
    docker ps
```

Command to Stop All Running Containers

```
docker stop $(docker ps -q)
```

- `docker ps -q`: Lists only the IDs of all running containers.
- This stops all running containers at once.

2. Remove Stopped Containers

Once stopped, containers still exist but are not running. You can remove them:

Command to Remove a Specific Container

```
docker rm <container_id_or_name>
```

Command to Remove All Stopped Containers

```
docker container prune
```

- This will prompt you for confirmation before removing all stopped containers.

3. Remove Docker Images

Images are separate from containers. Deleting images doesn't affect running containers, but you must remove containers that depend on those images before deleting them.

Command to Remove a Specific Image

```
docker rmi <image_id_or_name>
```

- Replace `<image_id_or_name>` with the image's ID or name, which you can find using:`bash`

```
docker images
```

Command to Remove All Unused Images

```
docker image prune
```

- Removes images not associated with any container.

Command to Force-Remove All Images

If you want to remove **all images** (be cautious with this):

```
docker rmi $(docker images -q)
```

- `docker images -q`: Lists only the IDs of all images.

4. Stop and Clean Up Everything

If you want to stop all containers, remove them, and then remove all images:

```
docker stop $(docker ps -q)               # Stop all running
containers
docker rm $(docker ps -aq)               # Remove all
containers (including stopped ones)
docker rmi $(docker images -q)           # Remove all images
```

5. Confirm Stopping or Removal

- **Check Running Containers**:

  ```
  docker ps
  ```

- **Check All Containers**:

  ```
  docker ps -a
  ```

- **Check Remaining Images**:

  ```
  docker images
  ```

Notes:

- **`stop`** is for containers; you can't "stop" an image since it's not running—it's just a file used to create containers.
- Use `docker container prune` and `docker image prune` to clean unused containers and images automatically.